PROYECTO TT 2026-B066 · SPRING BOOT

Checklist del backend FST

Qué está construido y qué falta, siguiendo el orden de construcción de la guía del proyecto (sección 12).

48%

69 de 145 casillas
hechas y verificadas en el código

AVANCE POR SECCIÓN

00	Base y Docker	10/10
01	Base de datos	6/8
02	Autenticación	9/11
03	Administración	0/7
04	Experimentos	9/16
05	Carga de video	11/11
06	Contrato con el worker	2/16
07	Resultados y revisión	4/17
08	Reportes	1/8
09	Notificaciones y correo	5/6
10	`worker_fake` y tareas programadas	0/11
11	Criterios de aceptación finales	1/8
12	Decisiones por tomar con el frontend	11/16

Rama poc/backend-spring · el backend compila · generado desde CHECKLIST.md

☐ Pendiente ☑ Hecho y verificado en el código La nota **Hecho:** en una casilla vacía dice qué parte ya existe

El SQL se verificó contra PostgreSQL el 15-oct. La sección 12 lista decisiones por tomar con el frontend (18-oct). No cubre el frontend ni el SQL y Docker de los anexos.

00 Base y Docker 10/10

- ☑ `docker compose up` levanta `db`, `backend`, `worker` y `mailhog` (dev), con volúmenes `videos_reportes` y `pgdata` (§2)
- ☑ `.env.example` con todas las variables de §2, más `VIDEO_TOKEN_MINUTES` (120). Las de autenticación y `ADMIN_*` el backend las lee desde la sección 2
- ☑ `application.properties` lee de variables de entorno lo que existe hoy (DB, SMTP, límite de subida)
- ☑ Endpoint `/health`
- ☑ Swagger con todas las rutas: sin `paths-to-match`, springdoc documenta también las que se agreguen (verificado en `/v3/api-docs`)
- ☑ CORS para el frontend (`localhost:5173`)
- ☑ Rutas guardadas en la BD relativas a la raíz del volumen, que en el contenedor es `/data/archivos` (§2): `videos/{idVideo}.{ext}`
- ☑ Límite de subida configurable, y `413` si se excede (§11.5): `ManejadorErrores` responde `{data: null, error, status: 413}`; probado con `MAX_UPLOAD_MB=1` y un archivo de 3 MB
- ☑ ffmpeg en el Dockerfile del backend (hace falta para §5); `ffprobe -version` corre en el contenedor
- ☑ Puerto del backend `8084`, dentro y fuera de Docker: el `8080` lo ocupa Traefik de CVAT en esta máquina. Falta avisarle a Vanesa (`VITE_API_BASE=http://localhost:8084`)

01 Base de datos 6/8

- ☑ SQL del Anexo B con los dominios cerrados, `UNIQUE` y llaves foráneas `NO ACTION` (§5), incluido `tipo_conducta`
- ☑ `ddl-auto=validate`
- ☑ 12 entidades, sin `Conducta`, `Intervalo` ni `Presenta` (§4.9, D-102)
- ☑ `ANALISIS.idVideo` único
- ☑ `rutaDiagnostico` solo se llena si `estado = 'error'` (`CHECK` en el SQL)
- ☐ Cardinalidades validadas en el servicio, sin `CHECK` de conteo: tanda de 2 a 4 especímenes, grupo con 1 o más tandas, mínimo 2 especímenes por grupo (§5)
- ☑ Contraseñas con bcrypt, nunca en texto plano: `BCryptPasswordEncoder` en el login y en `AdminInicial`, y el hash con `@JsonIgnore`

- [ ] Valores de dominio como enums de Java en lugar de `String` (opcional)

02 Autenticación

9/11

- [x] Spring Security con JWT de larga duración, configurable con `JWT_EXPIRES_HOURS` (§11.1): Nimbus HS256 con `tipo = sesion`, token solo en la cabecera (en la URL o alterado da `401`), y `ConvertidorJwt` revisa en la base en cada petición si la cuenta sigue activa (desactivada da `401` aunque el token no haya vencido) y si es Administrador. No arranca con un `JWT_SECRET` de menos de 32 caracteres
- [x] `POST /auth/login` con error `401` genérico (R-01), el mismo con contraseña mala, correo inexistente o cuenta inactiva. La respuesta trae `token`, `idInstitucional`, `nombre`, `apellidos`, `correo`, `rol` y `cambioRequerido`, sin el hash
- [x] `POST /auth/logout`, sin lista de bloqueo: pide token (§6) y responde `200`
- [x] `PATCH /auth/password`, y el cambio es obligatorio si `cambioRequerido`: mientras no la cambie, todo salvo esta ruta y el logout responde `403`; mínimo 8 caracteres y máximo 72 bytes (límite de bcrypt)
- [ ] `POST /auth/recover` envía un enlace JWT firmado por correo, con vigencia `RESET_LINK_MINUTES`
- [ ] `POST /auth/reset`
- [x] `PATCH /users/me` (perfil): nombre, apellidos y correo, `409` si el correo es de otra cuenta
- [x] Todo exige JWT salvo login, recover, reset, `/health` y Swagger: sin token se responde `401` con envoltorio; CORS centralizado y el preflight de `localhost:5173` pasa
- [x] Respuesta con envoltorio `{data, error, status}`, salvo las rutas que devuelven archivo (§6): `Respuesta.ok/error` en `/experiments` y `/health`, y `401` / `403` de Spring Security escritos con la misma forma
- [x] Manejo de errores global con `@RestControllerAdvice` (`ManejadorErrores` + `ErrorApi`) en lugar de mapas armados a mano: 400, 404, 405, 409, 413, 415 y 500, con el mensaje en `error`, sin `trace` y sin la causa en los 500
- [x] Comando para crear el primer administrador con `ADMIN_*`, que ponga `cambioRequerido = true` y lo inserte en `ADMINISTRADOR` (§11.7): `AdminInicial` al arrancar; idempotente (al reiniciar dice "ya existe" y no duplica), sin escribir la contraseña en el log; con `ADMIN_ID` vacío solo avisa

03 Administración

§6, R-07, R-16

0/7

- [ ] Verificación de rol Administrador en el backend en cada `/admin/*`, con `403` si no lo es
- [ ] `GET /admin/users` con filtros de rol y estado
- [ ] `POST /admin/users` genera una contraseña temporal, la envía por correo y la muestra una sola vez
- [ ] `409` si el identificador o el correo ya existen
- [ ] `PATCH /admin/users/{id}` modifica datos y rol, o desactiva con `activo = false`, nunca con `DELETE`

- [ ] `409` al desactivar la última cuenta de Administrador activa o quitarle el rol (R-16)
- [ ] `GET /admin/system` con disco y cola

04 Experimentos 9/16

- [x] `POST /experiments` con nombre único y fecha: `201`, `400`, `404`, `409` y `500`; el creador sale del JWT y se ignora el `idInstitucional` del cuerpo
- [ ] `GET /experiments` con filtros `estado`, `fecha` y `tratamiento`, paginado, con `nGrupos` y `avisoVencimiento`

  Hecho: `{pagina, total, items}` de 20 en 20, con `nGrupos`, `avisoVencimiento`, `creador` y los campos de la sección 12, en 4 consultas por página. Falta: los filtros en el servidor (hoy el frontend filtra en el navegador)
- [x] `GET /experiments/{id}` con el árbol de grupos, tandas y videos; cada video trae `fecha`, `videoDisponible`, `idAnalisis` y `subidoPor`. Limitación: `VIDEO` no guarda quién lo subió, así que `subidoPor` es el creador del experimento
- [x] El estado del experimento se calcula, no se guarda: `error` > `cancelado` > `en pausa` > `procesando` > `en cola` > `completado`, además de "sin videos" y "en espera del Día 2" (§4.8)
- [x] `GET /experiments/{id}/groups?q=` sugiere grupos y la siguiente tanda (sin distinguir mayúsculas ni acentos)
- [x] `PUT /experiments/{id}` con `404` y `409` por nombre repetido
  - [x] Nombre, fecha y notas; el creador no cambia
  - [x] `404` si no existe y `409` si el nombre ya es de otro
- [ ] `DELETE /experiments/{id}`: solo el creador o un Administrador (R-08)
  - [x] `404` si no existe
  - [ ] `403` si es otro usuario

    Hecho: implementado; se prueba en la sección 3, con una segunda cuenta
  - [ ] `409` si hay un análisis `procesando` o `en pausa`

    Hecho: `409` si tiene grupos (temporal, hasta tener el borrado completo)
  - [ ] Los `en cola` salen de la cola
  - [ ] Borra de las hojas hacia arriba, en una transacción: `SEGUNDO`, `OBSERVACION`, `ANALISIS`, `VIDEO`, `ESPECIMEN`, `TANDA`, `GRUPO`, `REPORTE`, `NOTIFICACION`, `EXPERIMENTO`

    Hecho: `REPORTE` y `NOTIFICACION`, en una transacción
  - [ ] Borra también la carpeta `analisis/{id}/` y los archivos
- [x] Todos los investigadores activos ven todos los experimentos (R-07)

05 Carga de video

§4.1, R-02, R-03, R-09

11/11

- [x] `POST /experiments/{id}/groups/{grupo}/videos` en `multipart/form-data` (probado hasta con 940 MB: unos 2 min, sin cargar el archivo en memoria)
- [x] Extensión `.mp4` o `.mov` revisada en el servidor
- [x] Reproducible y en horizontal, validado con ffprobe (con la marca de giro), y `422` con mensaje claro si falla
- [x] `409` si la tanda ya tiene un video de esa sesión
- [x] Crea el grupo si es nuevo, con `tipo` y `tratamiento`
- [x] Crea la tanda si es nueva, con `nCilindros` de 2 a 4 y `ordinal`
- [x] Crea los especímenes
- [x] Guarda el archivo en `videos/{idVideo}.{ext}`; un rechazo o una falla no deja archivos ni filas
- [x] Solo el Día 2 crea `ANALISIS` con `estado='en cola'`, `etapa='preprocesamiento'`, `nivelClasif='preciso'` y `fechaAnalisis=ahora` (D-79), y el worker real lo toma solo
- [x] El Día 1 se guarda sin análisis
- [x] Respuesta `201` con `{idVideo, idAnalisis, estadoAnalisis}`

06 Contrato con el worker

§4.2 a §4.8

2/16

- [ ] El backend nunca escribe `estado`, `etapa` ni `nivelClasif`, salvo al encolar, al reiniciar y en la pausa (§4.7)
- [x] `GET /experiments/{id}/status` con porcentaje de 0, 33, 67 o 100 por etapa. Es la única fuente del progreso: el frontend ya no usa una cola global
- [ ] Pausa:
  - [ ] `GET .../pause` lee `pausa.json` y calcula `segundosRestantes`
  - [ ] `GET .../pause/image`
  - [ ] `POST .../pause/adjust` escribe `ajuste` en el JSON y cambia `en pausa` a `en cola`, solo si sigue `en pausa` (`409` si no)
  - [ ] `POST .../cancel` cambia a `cancelado` con la misma condición
  - [ ] Ambas solo para quien subió el video o un Administrador
  - [ ] Actualización condicional atómica (`UPDATE ... WHERE estado='en pausa'`)
- [ ] `POST .../restart` (R-04):
  - [ ] Solo desde `completado`
  - [ ] Solo para quien subió el video o un Administrador

- [ ] En una transacción: borra `OBSERVACION` y `SEGUNDO`, borra los reportes y deja `en cola` con los valores de arranque
- [ ] `409` en `error`, `en pausa`, `cancelado` o en curso
- [ ] `409` si el video ya se borró (`VIDEO.archivo` nulo), porque reiniciar vuelve a correr el análisis sobre el video

- [x] `GET .../diagnostic` entrega el PDF de diagnóstico

07 Resultados y revisión 4/17

- [x] `GET .../results`, con totales y porcentaje por conducta y la bandera `revisado`
- [x] `GET .../results/byminute`: cada espécimen con sus minutos (las cuatro conductas siempre), desde `SEGUNDO`
- [x] `GET .../results/groups`, con promedio (no suma), datos faltantes y aviso si mezcla `preciso` y `agrupado`
- [ ] `GET .../results/blocks`, bloques de 5 s con la mayoritaria y el desempate entre nado y escalamiento por "conducta activa" (R-12)
- [x] Los totales se calculan desde `SEGUNDO`, nunca se guardan (D-52)
- [ ] `GET .../review`, con `puedeCorregir`, recuadros desde `cajas.json` y segundos
- [ ] `GET .../video` con soporte de `Range` y token de video (decidido el 15-oct)
  - [ ] `POST .../analyses/{idAnalisis}/video-token`: token firmado de solo lectura de ese análisis, con vigencia `VIDEO_TOKEN_MINUTES`; solo para quien subió el video o un Administrador, con el análisis `completado` y el video disponible [PROPUESTA]
  - [ ] `GET .../video` acepta `?vt=` solo en esta ruta y solo para ese análisis; el JWT de sesión no se acepta en la URL y el token de video no sirve en otras rutas
  - [ ] Sin query string en los logs de nginx ni de Spring
  - [ ] Si el video ya se borró, responde que ya no existe (`404`)
- [ ] `PUT .../segundos`:
  - [ ] Solo los segundos que cambiaron
  - [ ] `origen` en `confirmada`, `corregida` o `dudosa`
  - [ ] Solo el que subió el video o un Administrador
  - [ ] Solo sobre un análisis `completado`
  - [ ] `409` si el video ya se borró (`VIDEO.archivo` nulo): sin video no se corrige

08 Reportes

§7, R-06 — 1/8

- [ ] CSV con las columnas `Grupo, Tanda, Espécimen, Tiempo, Nado, Escalamiento, Inmovilidad, Conducta activa`
- [ ] Estadísticas por grupo: media, desviación estándar y varianza
- [ ] XLSX (Apache POI)
- [x] PDF de resultados con iText (`GET /experiments/{id}/report/pdf`), con los minutos contados desde `SEGUNDO`
- [ ] Se guardan en `REPORTE`
- [ ] Vigencia: si no existe, se genera; si es más viejo que el último análisis, se regenera y reemplaza; si no, se devuelve el guardado
- [ ] Indican si hubo revisión humana
- [ ] Los reportes se conservan aunque el video se borre

09 Notificaciones y correo

§11.2 — 5/6

- [x] `EmailService` genérico (destinatario, asunto, texto)
- [x] Si el correo falla, no se detiene la operación
- [x] `GET /notifications` y `PATCH /notifications/{id}` para marcar como leída. Cada notificación trae `tipo`, `mensaje`, `leido`, `fechaCreacion` e `idExperimento`; el frontend arma el título y el enlace
- [x] Solo las 6 notificaciones del dominio: las crea solo el worker; el backend no agrega otras
- [ ] Cada notificación se envía también por correo
- [x] Quitar el correo de demostración al crear un experimento, que no está en la guía

10 `worker_fake` y tareas programadas

0/11

> El `worker_fake` está en `../worker_fake` y es Python, no Spring. Ya escribe `OBSERVACION` y `SEGUNDO` con el modelo de 12 tablas. No lo he revisado punto por punto contra esta lista. Desde el 18-oct hay además un **worker real** (`../worker`, servicio `worker` del compose; `worker_fake` quedó en el perfil `fake`). Usa el modelo `fst_cnn_seg.pt` con el código de `fst_auto` copiado sin cambios en `../pipeline`. Probado con `AcuBN1.MOV`: completado, 4 × 300 segundos en `SEGUNDO`, `web.mp4`, `cajas.json`, PDF y notificación, y el camino de error con un archivo que no es video. Guarda en la base exactamente lo que da el modelo (100 %), pero el modelo en Docker coincide en 95.75 % de los segundos con el mismo modelo corrido en la PC (la diferencia nace al medir los fotogramas, antes de la red). No hace la pausa de R-15.

- [ ] Toma un análisis a la vez, el más antiguo primero

- [ ] Caso exitoso: pasa por las etapas, escribe `OBSERVACION` y `SEGUNDO` (últimos 300 s, renumerados) y crea la notificación
- [ ] Caso de error con la palabra `falla`: PDF de diagnóstico, estado `error` y notificación
- [ ] Caso de pausa con la palabra `pausa`, con sus tres salidas: ajuste, cancelación y vencimiento a los 5 minutos
- [ ] Al arrancar, reencola los `procesando` huérfanos
- [ ] Reemplaza `fechaAnalisis` por `ahora` al terminar
- [ ] Tareas diarias en un módulo aparte, disparables a mano:
  - [ ] Borrar videos vencidos a los 30 días
  - [ ] Aviso a 7 días
  - [ ] Vigilar el disco: alerta al 80 %, borrado al 90 %
- [ ] `ultima_corrida_tareas.txt`

11 Criterios de aceptación finales

§12 — **1**/8

- [ ] `docker compose up` arranca en una máquina limpia, y en una segunda máquina
- [ ] Un investigador recibe `403` en `/admin/*`
- [ ] No se puede quitar al último Administrador
- [x] Un Día 1 se guarda sin análisis
- [ ] Con dos análisis en cola, el worker los procesa uno a la vez
- [ ] Un reporte se regenera si hay un análisis más nuevo
- [ ] Cuatro sesiones simultáneas responden en 3 s o menos
- [ ] El resumen final anota los puntos [ABIERTO] que se resolvieron con el valor por defecto

12 Decisiones por tomar con el frontend

18-oct-2026 — **11**/16

> Vanesa conectó el frontend a la guía de la API (`PARA_ERICK.md`, puntos 41 a 52, rama `feat/frontend-v2-servicios`). Lo que sigue son formas que la guía no define y que ella propone. Cada casilla se marca cuando se decide y el endpoint correspondiente queda hecho. Sin respuesta, el frontend usa su propuesta y no se rompe.

- [x] `PATCH /auth/password` y `PATCH /users/me`: cuerpo `{contrasenaActual, contrasenaNueva}` y `{nombre, apellidos, correo}`; ambos devuelven la cuenta como el login, sin `token`
- [x] `POST /experiments` responde `201` con `{idExperimento}`

- [x] `GET /experiments`: campos extra `nEspecimenes`, `tratamientos`, `videosDia2: {analizados, cargados, total}`, `diasRetencion` (`null` sin videos o ya borrados) y `videosBorrados`
- [x] `GET /experiments/{id}/status`: incluye todos los análisis vigentes, también los `completado`
- [x] `/status`: `fechaAnalisis` es la hora de entrada a la cola mientras el análisis no termina (D-79); el frontend ordena la cola con ella
- [ ] `/status`: campos extra `pausa.desdeSegundo` y `motivoCancelacion` (`usuario` o `plazo`)

  Hecho: `pausa.desdeSegundo`. Falta `motivoCancelacion`, que no se guarda en ningún lado (sale `null`)
- [x] `GET /experiments/{id}`: cada video trae `fecha`; `/status` trae `fechaAnalisis` por análisis
- [ ] `GET /admin/system`: forma `{disco, conductas, cola: [{idAnalisis, idExperimento, experimento, grupo, letra, sesion, estado}]}`, en orden de llegada
- [x] `GET /experiments/{id}/groups?q=`: forma `[{idGrupo, etiqueta, tipo, tratamiento, siguienteOrdinal}]`
- [x] `{grupo}` en la URL de subida es la etiqueta del grupo, codificada (ver `services/groups.js` del frontend)
- [x] `/results`: cada espécimen trae además `idGrupo`
- [x] `/results/groups`: forma `[{idGrupo, etiqueta, tipo, tratamiento, n, nTotal, faltantes, nivelMixto, promedios, porNivel}]`
- [ ] `/results/blocks`: cada espécimen con `bloques: [{inicio, conducta, matiz, segundos}]`
- [ ] `GET .../review`: agregar `duracion`, `ancho` y `alto` (sin ellos el frontend asume 300 s y 1920 × 1080)
- [x] Notificaciones: el id se llama `idNotificacion`; «marcar todas» se hace con un `PATCH` por notificación, sin `POST /notifications/read-all`
- [ ] `POST .../video-token` responde `{token}`; probar `Range` con un `web.mp4` real